

Data Importation, Cleaning, and Wrangling

Week 2, Day 2: ML & AI Data Processing Pipeline

Victor Wandera Lumumba, MSc.

Statistician | Data Analyst | ML Expert

Mediacrest Training College

10th Oct 2026

Abstract

This presentation explores the end-to-end data processing pipeline using the General Social Survey (GSS) subset dataset. It covers the critical steps required to transform raw, messy data into a clean, analysis-ready format.

- **Data Importation & Inspection:** Loading libraries, reading CSVs, and understanding data structure, types, and distributions.
- **Data Cleaning:** Systematically handling missing values (imputation vs. deletion), removing duplicates, and detecting/capping outliers using the IQR method.
- **Data Manipulation:** Filtering rows based on complex conditions, selecting specific columns, and performing advanced group-by aggregations.
- **Feature Engineering:** Creating new categorical features (e.g., Income and Age brackets) from continuous variables to enhance downstream machine learning models.

Session Objectives

- Master the art of importing and inspecting datasets using `pandas` and `numpy`.
- Evaluate and apply appropriate strategies for handling missing data (Mean, Median, Mode, and Complete Cases).
- Detect and mitigate the impact of outliers using the Interquartile Range (IQR) capping technique.
- Perform advanced data manipulation including multi-condition filtering and complex aggregations.
- Engineer new features by binning continuous variables into meaningful categorical groups.

Part 1: Data Import & Inspection

"Understanding the raw data structure."

Data Wrangling Overview

- **The Reality of Raw Data:** Real-world datasets are rarely clean. They contain missing values, duplicates, extreme outliers, and inconsistent formatting.
- **What is Data Wrangling?** The process of cleaning, structuring, and enriching raw data into a desired format for better decision-making and faster computation.
- **Why is it Crucial for ML?** Machine learning algorithms are highly sensitive to data quality. "Garbage in, garbage out." Proper wrangling prevents biased models and skewed predictions.
- **The Pipeline:** Import $\rightarrow$ Inspect $\rightarrow$ Clean $\rightarrow$ Manipulate $\rightarrow$ Engineer Features.

Loading Required Libraries

Python Implementation

```
import pandas as pd            # Data importation and handling data frames
import numpy as np             # Handling numeric operations
import matplotlib.pyplot as plt # Data visualization
import seaborn as sn           # Advanced data visualization

### Filter warnings for cleaner output
import warnings
warnings.filterwarnings("ignore")

```

Code Explanation

- `pandas` is the backbone of data manipulation in Python.
- `numpy` provides fast numerical computations and array handling.
- `matplotlib` and `seaborn` are used for exploratory data visualization.

Reading the Dataset

Python Implementation

```
# Read the CSV file into a pandas DataFrame
df = pd.read_csv("GSSsubset1.csv")

```

Code Explanation

- The `pd.read_csv()` function loads the comma-separated values file into a 2D DataFrame.
- The dataset used is a subset of the General Social Survey (GSS), containing demographic and socioeconomic variables.

General Data Inspection: Shape & Columns

Python Implementation

```
# Check column names
print(f"Columns: {df.columns.tolist()}")
# Output: ['id', 'sex', 'degree', 'income', 'marital', 'age', 'height', 'weight', '
    hrswrk']

# Check the shape of the data
print(f"Shape: {df.shape}")
# Output: Shape: (3000, 9)

```

Data Explanation

- The dataset contains **3000 observations** (rows) and **9 variables** (columns).
- Variables include identifiers (`id`), demographics (`sex`, age, `marital`), socioeconomic status (`degree`, `income`), and physical metrics (`height`, `weight`, `hrswrk`).

Variable Types & Structure

Python Implementation

```
df.info()

```

Data Explanation

- **Data Types:** The dataset contains a mix of `float64` (4), `int64` (2), and `str/object` (3) variables.
- **Memory Usage:** The dataset is lightweight, consuming only 211.1 KB of memory.
- **Non-Null Counts:** Initial inspection reveals that columns like `income` (2767), `weight` (2879), and `hrswrk` (2933) have fewer non-null entries than the total 3000 rows, indicating missing data.

First & Last Observations

Python Implementation

```
# View the first 20 rows
df.head(20)

# View the last 10 rows
df.tail(10)

```

Data Explanation

- `head()` and `tail()` provide a quick sanity check of the data.
- We can visually confirm the presence of `NaN` (Not a Number) values in columns like `degree`, `income`, and `hrswrk`.
- Categorical variables like `sex` and `marital` are stored as uppercase strings (e.g., "MALE", "MARRIED").

General Descriptive Statistics

Python Implementation

```
# Remove the 'id' column from summary statistics
df.drop(columns=["id"]).describe()

```

Data Explanation

- **Income:** Mean is $63,871, but the median is $49,586. The max is $609,741. This indicates a right-skewed distribution (wealth concentration).
- **Age:** Ranges from 18 to 80, with a mean of 42 years.
- **Hours Worked (`hrswrk`):** Mean is $\sim$40 hours, aligning with a standard full-time work week.

General Frequency Tables

Python Implementation

```
# Distribution of Gender
print(df["sex"].value_counts())
# FEMALE: 1563, MALE: 1437

# Distribution of Education Qualification
print(df["degree"].value_counts())
# HIGH SCHOOL: 1094, LT HIGH SCHOOL: 583, BACHELOR: 566...

```

Data Explanation

- **Gender:** The dataset is relatively balanced (52% Female, 48% Male).
- **Education:** "High School" is the most common qualification, followed by "Less than High School".
- **Marital Status:** "Married" is the dominant category (1518 observations).

Part 2: Data Cleaning
Missing Values

"Strategies for handling incomplete data."

Checking the Presence of Missing Values

Python Implementation

```
# Absolute count of missing values
print(df.isna().sum())
# income: 233, weight: 121, hrswrk: 67, marital: 66, degree: 58

# Percentage of missing values
Missing_values = df.isna().mean().round(4)*100
print(Missing_values)
# income: 7.77%, weight: 4.03%, hrswrk: 2.23%, marital: 2.20%, degree: 1.93%

```

Data Explanation

`income` has the highest missingness at 7.77%. Since no column exceeds 10% missing data, we can safely apply imputation techniques rather than dropping the columns entirely.

Methods of Handling Missing Values

- **Complete Cases Analysis (Row Deletion):** Dropping any row with a missing value.
  - *Pros:* Simple, preserves data integrity.
  - *Cons:* Risky if data loss is significant; reduces sample size and statistical power.
- **Mean / Median Imputation:** Filling missing numeric values with the column's mean (symmetric data) or median (asymmetric/skewed data).
- **Mode Imputation:** Filling missing categorical/character data with the most frequent value.

Complete Cases Analysis (Row Deletion)

Python Implementation

```
# Drop all rows with any missing values
df_drop_all = df.dropna()

print(df_drop_all.info())
# Index: 2503 entries, 0 to 2999

```

Data Explanation

- Using `dropna()` reduces the dataset from 3000 to 2503 rows.
- We lose nearly 17% of our data. While this ensures complete cases, it may introduce bias if the missingness is not completely random (MCAR). We will prefer imputation to retain the full sample.

Mean Imputation (for Symmetric Data)

Python Implementation

```
# Identify numeric columns
numeric_cols = df_mean.select_dtypes(include="number").columns

# Fill missing values with the mean of each column
df_mean[numeric_cols] = df_mean[numeric_cols].fillna(df_mean[numeric_cols].mean())

```

Data Explanation

- Mean imputation is best for variables that follow a normal (symmetric) distribution.
- It preserves the sample size but can artificially reduce the variance of the dataset.

Median Imputation (for Asymmetric Data)

Python Implementation

```
# Identify numeric columns
numeric_cols = df_median.select_dtypes(include="number").columns

# Fill missing values with the median of each column
df_median[numeric_cols] = df_median[numeric_cols].fillna(df_median[numeric_cols].
    median())

```

Data Explanation

- Median imputation is robust against outliers and is ideal for skewed distributions (like `income`).
- Unlike the mean, the median is not pulled by extreme values, providing a more representative central tendency for imputation.

Mode Imputation (Categorical Data)

Python Implementation

```
# Identify categorical columns
cat_cols = df.select_dtypes(include=["object", "category"]).columns

for col in cat_cols:
    mode_val = df[col].mode()
    if not mode_val.empty:
        df[col] = df[col].fillna(mode_val[0])

```

Data Explanation

- For categorical variables like `degree` and `marital`, we use the mode (most frequent category).
- For example, missing `degree` values are filled with "HIGH SCHOOL", and missing `marital` values are filled with "MARRIED".

Verifying Imputation

Python Implementation

```
# Check missing values after imputation
print(df.isna().sum())
# Output: All columns show 0 missing values

# Check data structure
df.info()
# Output: 3000 entries, 0 non-null across all columns

```

Data Explanation

After applying mode imputation for categorical variables and mean imputation for numeric variables, the dataset is fully complete. No rows need to be dropped, preserving the full sample size of 3000 observations for modeling.

Part 3: Data Cleaning
Duplicates & Outliers

"Removing noise and extreme anomalies."

Handling Duplicate Entries

Python Implementation

```
# Check for duplicate rows
dup_rows = df.duplicated().sum()
print(dup_rows) # Output: 0

# Drop duplicates if any exist
clean_df = df.drop_duplicates()
clean_df.info()

```

Data Explanation

- Duplicates can arise from data entry errors or merging issues.
- In this dataset, there are 0 duplicate rows. However, running `drop_duplicates()` is a standard best practice to ensure model training isn't biased by repeated observations.

Introduction to Outliers

- **Definition:** Outliers are data points that lie abnormally very low or very high, far off the general trend of the distribution.
- **Causes:** Measurement errors, data entry typos, or genuine extreme variations in the population.
- **Impact on ML:**
  - Skew the mean and standard deviation.
  - Disproportionately influence regression coefficients and distance-based algorithms (e.g., KNN, K-Means).
  - Can cause tree-based models to create overly complex branches to accommodate extreme values.

Detecting Outliers: The IQR Method

Python Implementation

```
numeric_cols = df.select_dtypes(include=["float64", "int64"]).columns
for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]
    print(f"Columns '{col}': {len(outliers)} Outliers")

```

Identifying Outlier IDs

Python Implementation

```
numeric_cols = ["income", "age", "height", "weight", "hrswrk"]
outlier_id = {}
for col in numeric_cols:
    # ... calculate Q1, Q3, IQR, bounds ...
    outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]
    outlier_id[col] = outliers["id"].tolist()
    print(f"Column '{col}': {len(outliers)} outliers")

```

Graph & Data Explanation

- **Income:** 201 outliers (extreme high earners).
- **Hours Worked (`hrswrk`):** 155 outliers.
- **Weight:** 32 outliers.
- **Age & Height:** Very few outliers (10 and 2 respectively).

Capping the Outliers

Python Implementation

```
numeric_cols = df.select_dtypes(include=["float64", "int64"]).columns
for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    ## Cap the Outliers
    df[col] = np.where(df[col] < lower_bound, lower_bound, df[col])
    df[col] = np.where(df[col] > upper_bound, upper_bound, df[col])

```

Code Explanation

Instead of deleting outlier rows, we **cap** them. Values below the lower bound are set to the lower bound, and values above the upper bound are set to the upper bound. This preserves the sample size while mitigating extreme skew.

Post-Capping Verification & Saving

Python Implementation

```
# Verify no outliers remain
for col in numeric_cols:
    # ... recalculate bounds ...
    outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]
    print(f"Columns '{col}': {len(outliers)} Outliers")
# Output: All columns show 0 Outliers

# Save the cleaned dataset
df.to_csv("Cleaned_gss_data.csv", index=False)

```

Data Explanation

The capping process successfully neutralized all extreme values. The cleaned dataset is now exported to a new CSV file, ready for the manipulation and modeling phases.

Part 4: Data Manipulation Filtering & Selection

"Subsetting the data for targeted analysis."

Data Manipulation Overview

- **Filtering:** Subsetting rows based on specific logical conditions.
- **Selection:** Extracting specific columns of interest.
- **Grouping & Aggregating:** Splitting data into groups and applying summary statistics.
- **Combining:** Merging filtering and selection to create highly specific analytical views.

Note

For the manipulation phase, we reload the dataset to demonstrate operations on the raw structure before final feature engineering.

Filtering: Single Condition

Python Implementation

```
# Create male participants data
male = df[df['sex'] == 'MALE']
male.head()

```

Code Explanation

- We use boolean indexing to filter rows.
- The condition `df['sex'] == 'MALE'` creates a boolean mask.
- Passing this mask inside `df[]` returns only the rows where the condition evaluates to `True`.

Filtering: Multiple Conditions (AND)

Python Implementation

```
subset = df[(df['sex'] == 'FEMALE') &
    (df['age'] < 35) &
    (df['marital'] == "MARRIED") &
    (df['degree'] == "GRADUATE")]
subset.head()

```

Code Explanation

- The & operator acts as a logical AND.
- **Crucial Syntax:** Each condition must be enclosed in parentheses () due to Python's operator precedence rules.
- This subset isolates young, married, highly educated females.

Filtering: OR Condition

Python Implementation

```
high_educ = df[(df['degree'] == "BACHELOR") |
    (df['degree'] == "GRADUATE")]
high_educ.head(20)

```

Code Explanation

- The | operator acts as a logical OR.
- This filter captures anyone with either a Bachelor's or a Graduate degree.
- Alternatively, the `.isin()` method can be used for cleaner syntax when checking against multiple categories: `df['degree'].isin(["BACHELOR", "GRADUATE"])`.

Column Selection & Dropping

Python Implementation

```python
# Option 1: Select specific columns
cols = ["sex", "age", "income", "marital"]
selected_cols = df[cols]

# Option 2: Drop unwanted columns
drop_cols = df.drop(columns=["id", "degree", "height", "hrswrk", "weight"])

```

Code Explanation

- Selecting specific columns reduces dimensionality and focuses the analysis on relevant features.
- `drop()` is useful when you want to keep most columns but exclude a few (like identifiers or irrelevant metrics).

Part 5: Data Manipulation Grouping & Aggregation

"Extracting insights through summarization."

Group By and Summarize

Python Implementation

```
# Mean Income by Education Levels
income_by_educ = df.groupby('degree')['income'].mean().sort_values(ascending=False)
print(income_by_educ)

# Mean Income by Marital Status
income_by_marital = df.groupby('marital')['income'].mean().sort_values(ascending=
    False)

```

Data Explanation

- **Education:** "GRADUATE" degree holders have the highest mean income ($68,457), while "LT HIGH SCHOOL" has the lowest ($18,412).
- **Marital Status:** "MARRIED" individuals report the highest mean income, followed by "DIVORCED".

Multiple Grouping & Resetting Index

Python Implementation

```
# Group by Sex and Degree
income_by_educ = df.groupby(['sex', 'degree'])['income'].mean()

# Reset the index to convert back to a standard DataFrame
income_by_educ = df.groupby(['sex', 'degree'])['income'].mean().reset_index()
income_by_educ.head()

```

Code Explanation

- Passing a list of columns to `groupby()` creates a MultiIndex (hierarchical index).
- `reset_index()` flattens the MultiIndex, turning the grouped categories back into standard columns, which is essential for downstream plotting or modeling.

Advanced Grouping and Multiple Aggregation

Python Implementation

```
summary = df.groupby('degree').agg({
    'income': ['mean', 'median', 'std', 'var', 'count'],
    'age': ['mean', 'median'],
    'height': ['mean', 'median']
})
summary

```

Code Explanation

- The `.agg()` method allows us to apply multiple statistical functions to different columns simultaneously.
- This generates a comprehensive summary table showing the central tendency and dispersion of income, age, and height across all education levels.

Part 6: Feature Engineering

"Creating new insights from existing data."

Feature Engineering: Creating Categories

Python Implementation

```
# 1. Income Categories based on Mean
mean_income = df['income'].mean()
df["Income_Category"] = np.where(df["income"] >= mean_income,
                                 "HIGH INCOME", "LOW INCOME")

# 2. Age Categories using pd.cut
bins = [19, 35, 45, 55, float('inf')]
labels = ["YOUNG", "YOUNG ADULT", "MIDDLE ADULT", "OLDER ADULTS"]
df['Age_Cat'] = pd.cut(df['age'], bins=bins, labels=labels, right=False)

```

Code Explanation

- **Binning Continuous Variables:** Converts raw numeric values into meaningful categorical groups.
- **Income:** Split into "HIGH" and "LOW" based on the dataset mean ($36,887).
- **Age:** Split into 4 demographic cohorts for targeted analysis.

Final Conclusion & Next Steps

- **Data Quality is Paramount:** Rigorous cleaning (imputation, outlier capping) ensures that machine learning models learn true patterns rather than noise.
- **Manipulation Unlocks Insights:** Grouping and filtering allow analysts to uncover demographic trends (e.g., income disparities by education and gender).
- **Feature Engineering Adds Value:** Transforming continuous variables into categorical bins can help non-linear models capture demographic thresholds more effectively.
- **Next Steps:** The cleaned and engineered dataset is now ready for exploratory data visualization and subsequent predictive modeling.

Victor Wandera Lumumba
Statistician | Data Analyst | ML Expert
Mediacrest Training College